

Sieci komputerowe - ściągą na laby i kolokwium praktyczne

Linux/Cisco/VLAN/iptables/NAT. Bez WiFi. Najpierw adresacja, potem kable, adresy, routing, filtry/NAT, testy.

Plan działania na kolokwium 45 min

1. **Przerysuj schemat i nadaj nazwy interfejsom.** Zaznacz, które urządzenie jest routerem, które sieci są rozłączne, gdzie ma być NAT/VLAN/firewall.
2. **Adresacja daje dużo punktów.** Dla każdej domeny L2/VLAN/łącza punkt-punkt wybierz osobną sieć prywatną. Najwygodniej: 10.X.Y.0/24, dla łączy P2P można /30 albo też /24, jeśli nie ma wymogu oszczędzania.
3. **Połącz fizycznie.** PC-switch i switch-router: prosty; PC-PC, router-router, switch-switch: krosowany, chyba że Auto MDI-X zadziała.
4. **Wyczyść stare rzeczy.** Usuń stare IP/routing/iptables, podnieś interfejsy.
5. **Nadaj adresy i sprawdź sąsiadów.** Najpierw pingi tylko między bezpośrednio połączonymi urządzeniami.
6. **Włącz forwarding na Linux-routerach.** Potem dodaj trasy statyczne lub defaulty.
7. **Dopiero na końcu iptables/NAT/VLAN.** Zawsze w skrypcie: flush, polityki, reguły.
8. **Udowodnij działanie.** Ping/traceroute/ssh/ncat/Wireshark/liczniki iptables.

Minimalny algorytm debugowania: adresacja rozłączna? właściwe porty/kable? interfejsy UP, LOWER_UP? właściwe maski? brak starych adresów? ping do sąsiada? forwarding? trasa w obie strony? pakiet widać w Wiresharku na każdym hoku? iptables puste albo poprawne? usługa słucha na właściwym IP, nie tylko na 127.0.0.1?

Sprzęt w laboratorium

PC i patch panel

- em1 - główny interfejs, zwykle Internet/DHCP, w mostku br0; na patch panelu zwykle 4. gniazdo.
- p4p1 - port od okna/prawy; zwykle 1. gniazdo.
- p4p2 - port od drzwi/lewy; zwykle 2. gniazdo, czasem niepodpięty.
- /dev/ttyS0 - port szeregowy do konsoli Cisco; zwykle 3. gniazdo, płaski niebieski kabel.

```
# identyfikacja karty - miga dioda
ethtool -p p4p1
ethtool p4p1
ip link show
```

Cisco przez konsolę

```
picocom /dev/ttyS0 # zwykle 9600, stare/czarne
picocom -b 115200 /dev/ttyS0 # nowe/białe switche
# wyjście: Ctrl-a, potem q
```

Jeśli po Enter nic nie ma: zły patch panel, zły kabel, zły baudrate, urządzenie nie włączone.

Adresacja IPv4, podsieci, agregacja

Podstawy

Adres IPv4 = 32 bity. Maskę /n mówi, ile bitów to sieć. Hosty w zwykłej sieci: $2^{32-n} - 2$ (bez adresu sieci i broadcastu). Adres sieci = IP AND maska. Broadcast = część hosta wypełniona jedynekami.

Pref.	Maska	Adresy	Hosty
/0	0.0.0.0	4 294 967 296	4 294 967 294
/1	128.0.0.0	2 147 483 648	2 147 483 646
/2	192.0.0.0	1 073 741 824	1 073 741 822
/3	224.0.0.0	536 870 912	536 870 910
/4	240.0.0.0	268 435 456	268 435 454
/5	248.0.0.0	134 217 728	134 217 726
/6	252.0.0.0	67 108 864	67 108 862
/7	254.0.0.0	33 554 432	33 554 430
/8	255.0.0.0	16 777 216	16 777 214
/9	255.128.0.0	8 388 608	8 388 606
/10	255.192.0.0	4 194 304	4 194 302
/11	255.224.0.0	2 097 152	2 097 150
/12	255.240.0.0	1 048 576	1 048 574
/13	255.248.0.0	524 288	524 286
/14	255.252.0.0	262 144	262 142
/15	255.254.0.0	131 072	131 070
/16	255.255.0.0	65 536	65 534
/17	255.255.128.0	32 768	32 766
/18	255.255.192.0	16 384	16 382
/19	255.255.224.0	8 192	8 190
/20	255.255.240.0	4 096	4 094
/21	255.255.248.0	2 048	2 046
/22	255.255.252.0	1 024	1 022
/23	255.255.254.0	512	510
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/29	255.255.255.248	8	6
/30	255.255.255.252	4	2
/31	255.255.255.254	2	0*
/32	255.255.255.255	1	1*

Szybkie liczenie podsieci

- Dla /n rozmiar bloku w ostatnim zmiennym okcie to 256 – wartość maski w tym okcie.
- Przykład /28: maska 240, blok 16: sieci kończą się na .0, .16, .32, .48, ... Hosty w 192.168.1.32/28: .33–.46, broadcast .47.
- Podział na p równych podsieci: pożycz $x = \lceil \log_2(p) \rceil$ bitów, nowa maska = stara + x .
- VLSM: sortuj wymagania od największej liczby hostów, przydzielaj kolejne najmniejsze pasujące bloki.
- Adresy prywatne: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.

Agregacja tras

Agreguj tylko wtedy, gdy nadsieć nie obejmie cudzych/nieistniejących sieci, chyba że zadanie na to pozwala. Szukaj najdłuższego wspólnego prefiksu. Przykład: 192.168.0.0/24 i 192.168.1.0/24 = 192.168.0.0/23. W praktyce na kolokwium często można zamiast wielu tras dać **default route** do najbliższego routera.

Linux: interfejsy, IP, routing

Czyszczenie i konfiguracja IP

```
ip addr show
ip link show
ip link set p4p1 up
ip link set p4p1 down
ip addr flush dev p4p1
ip addr add 192.168.10.2/24 dev p4p1
ip addr add 10.0.0.2/24 dev p4p1 label p4p1:1 # dodatkowy adres
ip addr del 192.168.10.2/24 dev p4p1
ip link set p4p1 address aa:bb:cc:dd:ee:ff
ethtool -P p4p1 # permanent MAC
```

Czytaj ip addr: nazwy interfejsów, MAC, UP/DOWN, adresy i sieci. Jeśli jest DOWN, podnieś. Jeśli brak LOWER_UP/lampki, sprawdź kabel/port.

Routing w Linuxie

```
ip route show
ip route get 8.8.8.8
ip route add 172.16.0.0/16 via 192.168.1.1
ip route del 172.16.0.0/16
ip route add default via 192.168.1.1
ip route replace default via 192.168.1.1
ip route flush cache # rzadko potrzebne
```

Czytaj ip route:

- wpis bez via: sieć bezpośrednio podłączona; src to adres używany jako źródłowy;
- wpis z via: pakiet idzie do bramy; brama musi leżeć w sieci bezpośrednio podłączonej;
- default: trasa ostatniej szansy;
- wybór trasy: najdłuższa maska > niższy metric > pierwszy pasujący wpis.

Forwarding - Linux jako router

```
cat /proc/sys/net/ipv4/ip_forward
echo 1 > /proc/sys/net/ipv4/ip_forward
sysctl net.ipv4.ip_forward
sysctl -w net.ipv4.ip_forward=1
# alternatywnie w niektórych zadaniach:
sysctl -w net.ipv4.conf.all.forwarding=1
```

Bez forwardingu Linux przyjmie pakiety do siebie, ale nie przekaże ich dalej. Forwarding jest niezależny od NAT.

ARP i DHCP

```
ip neigh show
ip neigh flush dev p4p1
ip neigh add 192.168.1.10 lladdr aa:bb:cc:dd:ee:ff dev p4p1
ip neigh del 192.168.1.10 dev p4p1
arping -I p4p1 192.168.1.10
ip link set arp off dev p4p1
ip link set arp on dev p4p1

# DHCP, jeśli trzeba odnowić em1/p4p1
dhclient -r -v em1
dhclient -v em1
journalctl -f
cat /etc/resolv.conf
cat /var/lib/dhclient/dhclient.leases # czasem /var/lib/dhcp/...
```

ARP działa tylko w lokalnej domenie L2/VLAN. Jeśli pingujesz host z innej sieci, ARP powinien być do bramy, nie do hosta docelowego.

Cisco routery: tryby, adresy, routing

Powłoka i diagnostyka

```
Router> enable
Router# configure terminal
Router(config)# hostname R1
R1(config)# end # albo Ctrl-Z
R1# show running-config
R1# show interfaces
R1# show interface GigabitEthernet 0/1
R1# show ip interface brief # nie pokazuje secondary!
R1# show ip route
R1# show cdp neighbors
R1# ping 192.168.1.2
R1# traceroute 192.168.3.2 numeric
```

Skróty: ?, Tab, do show . . . w trybie konfiguracji, no <komenda> odwraca działanie, no shutdown podnosi interfejs.

Adresy na interfejsach

```
R1# conf t
R1(config)# interface GigabitEthernet 4
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# interface GigabitEthernet 5
R1(config-if)# ip address 10.0.0.1 255.255.255.252
R1(config-if)# no shutdown
R1(config-if)# end
```

Routery z labu: Gi4/Gi5 są zwykłymi portami L3. Gi0-Gi3 mogą być za wewnętrznym switchem; wtedy adres nadaje się na interface Vlan1, a kabel można wpiąć w dowolny Gi0-Gi3.

Kilka adresów bez VLANów: secondary

```
R1# conf t
R1(config)# interface FastEthernet 0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# ip address 172.16.13.13 255.255.128.0 secondary
R1(config-if)# ip address 10.10.10.10 255.255.240.0 secondary
R1(config-if)# no shutdown
R1(config-if)# do show ip interface FastEthernet 0/0
```

Uwaga: show ip interface brief pokaże tylko adres główny, nie secondary.

Routing statyczny Cisco

```
R1# show ip route
R1# conf t
R1(config)# ip route 172.16.0.0 255.255.0.0 192.168.1.111
R1(config)# ip route 0.0.0.0 0.0.0.0 192.168.1.1
R1(config)# no ip route 172.16.0.0 255.255.0.0 192.168.1.111
```

Pakiet zwrotny też musi mieć trasę. Jeśli ping działa tylko w jedną stronę w Wiresharku, prawie zawsze brakuje routingu powrotnego albo filtr/NAT go zmienia.

OSPF Cisco - gdy pojawi się routing dynamiczny

```
R1# conf t
R1(config)# router ospf 1
R1(config-router)# network 10.0.0.0 0.0.255.255 area 0
R1(config-router)# auto-cost reference-bandwidth 1000
R1(config-router)# default-information originate # tylko gdy jest default
R1(config-router)# exit
R1(config)# interface GigabitEthernet 0/1
R1(config-if)# bandwidth 25000 # 25 Mb/s; IOS zwykle podaje kb/s
R1(config-if)# ip ospf cost 17 # reczny koszt OSPF, nadpisuje bandwidth
```

```
R1# show ip ospf neighbor
R1# show ip protocols
R1# show ip ospf interface
R1# show ip ospf database
R1# show ip route
```

Wildcard mask w network: odwrotność maski. /24 = 0.0.0.255, /16 = 0.0.255.255. OSPF ma AD 110, trasa statyczna AD 1 wygra z OSPF.

VLANy i router-on-a-stick

Zasady

VLAN = osobna domena rozgłoszeniowa w warstwie 2. Hosty w różnych VLANach nie komunikują się bez routera, nawet jeśli wpiszesz im adresy z tej samej sieci IP. Połączenie przenoszące wiele VLANów to **trunk**; ramki dostają tag 802.1Q. Port do zwykłego PC jest **access**.

Najpierw rozpoznaj porty

Nie ucz się numerów portów na pamięć. Najpierw sprawdź, jak urządzenie nazywa porty, a potem podstaw właściwe nazwy w miejsce <PORT_PC> i <PORT_TRUNK>.

```
S1# show ip interface brief
S1# show interface status
S1# show interfaces status
S1# show vlan
S1# show vlan brief
S1# show vlan-switch
S1# show interfaces trunk
```

Jeśli show vlan jest niejednoznaczne, wpisz show vlan ?. Na części starszych Cisco/EtherSwitch właściwa komenda to show vlan-switch; na części nowszych wystarcza show vlan albo show vlan brief. W wyjściu nowych/białych switchy porty tagged zwykle oznaczają trunk, a untagged port access.

Tworzenie VLANów: normalny IOS

```
S1# conf t
S1(config)# vlan <VLAN_ID>
S1(config-vlan)# name <NAZWA>
S1(config-vlan)# exit
S1(config)# end
```

vlan <VLAN_ID> tworzy albo wybiera VLAN. name nadaje tylko etykietę dla człowieka; separację robi numer VLAN. Domyślnie VLAN jest aktywny.

Tworzenie VLANów: starszy vlan database

Na niektórych starych/czarnych switchach vlan database działa z trybu uprzywilejowanego S1\#, a nie z conf t. W tym trybie end może być błędem; wychodzisz przez exit, co zapisuje zmiany.

```
S1> enable
S1# vlan database
S1(vlan)# vlan <VLAN_ID> name <NAZWA>
S1(vlan)# exit
```

Po utworzeniu VLANów wracasz do zwykłego conf t i dopiero tam przypisujesz porty.

Port access dla zwykłego hosta

```
S1# conf t
S1(config)# interface <PORT_PC>
S1(config-if)# switchport mode access
S1(config-if)# switchport access vlan <VLAN_ID>
S1(config-if)# no shutdown
S1(config-if)# end
```

interface <PORT_PC> wybiera port z listy show switchport mode access wymusza tryb jednego, nietagowanego VLANu. switchport access vlan <VLAN_ID> przypisuje ten port do VLANu hosta; samo nie robi trunku.

Port trunk między switchami albo do routera

```
S1# conf t
S1(config)# interface <PORT_TRUNK>
S1(config-if)# switchport trunk encapsulation dot1q
S1(config-if)# switchport mode trunk
S1(config-if)# no shutdown
S1(config-if)# end
S1# show interfaces trunk
```

switchport mode trunk przenosi wiele VLANów przez jeden kabel. switchport trunk encapsulation dot1q jest potrzebne tylko na starszych urządzeniach, które pozwalają wybrać enkapsulację; jeśli komenda jest niezna, zwykle dot1q jest jedyną opcją i przechodzisz dalej.

Router między VLANami

Port router-switch też musi być trunk. Na routerze nie nadajesz IP na fizycznym interfejsie, tylko na podinterfejsach.

```
R1# conf t
R1(config)# interface <PORT_D0_SWITCHA>
R1(config-if)# no ip address
R1(config-if)# no shutdown
R1(config-if)# interface <PORT_D0_SWITCHA>.<VLAN_A>
R1(config-subif)# encapsulation dot1Q <VLAN_A>
R1(config-subif)# ip address <BRAMA_A> <MASKA_A>
R1(config-subif)# interface <PORT_D0_SWITCHA>.<VLAN_B>
R1(config-subif)# encapsulation dot1Q <VLAN_B>
R1(config-subif)# ip address <BRAMA_B> <MASKA_B>
R1(config-subif)# end
R1# show ip interface brief
```

encapsulation dot1Q <VLAN> mówi, jaki tag 802.1Q obsługuje dany podinterfejs. Adres IP podinterfejsu jest bramą domyślną dla hostów w tym VLANie. Jeśli dwa switchy przenoszą VLANy, wszystkie połączenia między nimi muszą być trunkami.

Jak udowodnić VLANy

- Hosty w tym samym VLANie pingują się przez switchy.
- Hosty w różnych VLANach bez routera nie pingują się.
- Wireshark na porcie access nie widzi broadcastów z innych VLANów.
- Wygeneruj broadcast: `arping -I p4p1 <nieużyty-adres-w-mojej-sieci>` albo ping do nieużytego adresu w swojej podsieci, bo host wyśle ARP broadcast.

Warstwa transportowa i ncat do testów

Porty i procesy

```
netstat -tunap
netstat -tunap | grep ssh
ss -tunap
cat /etc/services | grep -w 74
systemctl status sshd
systemctl start sshd
ssh -v user@192.168.1.10
ssh -vv user@192.168.1.10
```

Porty: 1-1023 uprzywilejowane, 1024-49151 zarejestrowane, 49152-65535 efemeryczne. TCP ma stany LISTEN, SYN-SENT, ESTABLISHED, TIME-WAIT; UDP jest bezpołączeniowy.

Ncat: symulacja usług

```
# TCP server i klient
ncat -l 192.168.1.55 1234
ncat 192.168.1.55 1234

# UDP
ncat -ul 192.168.1.55 1234
ncat -u 192.168.1.55 1234

# zapis/odczyt pliku
ncat -l 192.168.1.55 1234 > out.txt
ncat 192.168.1.55 1234 < in.txt

# wiele połączeń i echo
ncat -lk 192.168.1.55 1234 -c 'while true; do read i && echo [echo] $i; done'

# prosty HTTP
ncat www.cs.put.poznan.pl 80
GET /tkobus/ HTTP/1.0
```

Uwaga: Do testów DNAT usługa musi słuchać na adresie osiągalnym z sieci, np. 192.168.111.1, a nie tylko na 127.0.0.1.

iptables: model mentalny

Netfilter, tabele i łańcuchy

Tabela	Do czego
filter	firewall: przepuszcza/blokuje pakiety. Łańcuchy: INPUT, FORWARD, OUTPUT.
nat	translacja adresów/portów. Łańcuchy: PREROUTING, OUTPUT, POSTROUTING.
mangle	modyfikacje pakietu, np. TTL. Łańcuchy: PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING.
raw	przed conntrackiem, rzadko na SK.

Przepływ pakietu przez Linuxa

- Do samego Linuxa: PREROUTING -> routing -> INPUT.
- Wygenerowany lokalnie: OUTPUT -> routing -> POSTROUTING.
- Przekazywany przez router: PREROUTING -> routing -> FORWARD ->

POSTROUTING.

Z tego wynika: INPUT nie filtruje ruchu przechodzącego przez router; do tego jest FORWARD. PREROUTING nie zna jeszcze interfejsu wyjściowego -o. POSTROUTING nie używa -i. DNAT zwykle robi się w PREROUTING, SNAT/MASQUERADE w POSTROUTING.

Najważniejsze zasady

- Reguły są sprawdzane od góry. Pierwsza decydująca reguła kończy przetwarzanie danego łańcucha.
- DROP ignoruje pakiet; klient czeka/timeout. REJECT odrzuca aktywnie, zwykle szybciej widać błąd; w Wiresharku pojawi się odpowiedź ICMP albo TCP RST zależnie od protokołu/opcji.
- Polityka -P działa, gdy żadna reguła nie pasuje. Na kolokwium blokowanie całego ruchu wejściowego rób polityką: iptables -P INPUT DROP.
- NAT korzysta z conntracka: odpowiedzi są automatycznie odtłumaczane. Reguła NAT zwykle trafia tylko pierwszy pakiet połączenia; potem działa stan połączenia.
- Dla routera z firewallem pamiętaj o FORWARD; przy polityce FORWARD DROP musisz dopuścić ruch routowany i powrotny.

iptables: składnia, reset, liczniki

Składnia i operacje

```
iptables [-t tabela] OPCJA LANCUCH [match...] -j CEL

# listowanie
iptables -L
iptables -t nat -L
iptables -t filter -L -n -v -x --line-numbers
iptables -S
iptables -t nat -S
iptables-save

# polityki
iptables -P INPUT DROP
iptables -P FORWARD ACCEPT
iptables -P OUTPUT ACCEPT

# dodawanie/usuwanie/czyszczenie
iptables -A INPUT ...      # append na koniec
iptables -I INPUT 1 ...    # insert na pozycje 1
iptables -D INPUT 3        # usun 3. regule
iptables -D INPUT ...      # usun regule po tresci
iptables -F INPUT          # flush lancucha
iptables -F                # flush filter
iptables -t nat -F         # flush nat
iptables -X                # usun własne lancuchy
iptables -Z                # wyzeruj liczniki
```

Bezpieczny szkielet skryptu

```
#!/bin/bash
set -e

# 1. śćWyczy. Najpierw polityki ACCEPT, zeby nie odciac sie podczas pracy.
iptables -P INPUT ACCEPT
iptables -P FORWARD ACCEPT
iptables -P OUTPUT ACCEPT
iptables -F
iptables -t nat -F
iptables -t mangle -F
iptables -X
iptables -Z

# opcjonalnie IPv6, zeby nie mieszało w testach IPv6
ip6tables -P INPUT ACCEPT 2>/dev/null || true
ip6tables -P FORWARD ACCEPT 2>/dev/null || true
ip6tables -P OUTPUT ACCEPT 2>/dev/null || true
ip6tables -F 2>/dev/null || true

# 2. Polityki docelowe
iptables -P INPUT DROP
iptables -P FORWARD ACCEPT
iptables -P OUTPUT ACCEPT

# 3. Reguly od najbardziej szczegolowych do ogolnych
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

Uwaga: Jeśli pracujesz lokalnie w labie, nie ma ryzyka utraty SSH do własnego PC, ale kolejność nadal ma znaczenie.

iptables: match-e i cele

Najczęstsze dopasowania

```
-s 192.168.1.0/24      # source
-d 10.0.0.5           # destination
-i p4pl               # interfejs wejscowy
-o em1               # interfejs wyjscowy
-p tcp               # tcp/udp/icmp
-p tcp --dport 22     # port docelowy
-p tcp --sport 12345  # port zrodlowy
-p udp --dport 53
-p icmp --icmp-type echo-request
! -s 192.168.1.55     # negacja
-m mac --mac-source aa:bb:cc:dd:ee:ff
-m owner --gid-owner 1006
-m comment --comment "opis reguly"
```


Conntrack - filtr stanowy

```
# wpuszczaj odpowiedzi na połączenia zainicjowane przez nas
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# na routerze z FORWARD DROP: pozwól sieci LAN wychodzić i odpowiedziom wracać
iptables -P FORWARD DROP
iptables -A FORWARD -i p4p1 -o em1 -s 192.168.1.0/24 -m conntrack --ctstate NEW,ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -i em1 -o p4p1 -d 192.168.1.0/24 -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

Stany: NEW - nowy przepływ, ESTABLISHED - część znanego połączenia, RELATED - powiązane, np. część ICMP błędów, INVALID - śmieci/niepasujące.

Connlimit - limit SSH

```
# maksymalnie 2 połączenia SSH z jednego adresu IP klienta
iptables -A INPUT -p tcp --dport 22 -m connlimit --connlimit-above 2 -j DROP

# globalnie maksymalnie 2 połączenia SSH łącznie
iptables -A INPUT -p tcp --dport 22 -m connlimit --connlimit-above 2 --connlimit-mask 0 -j DROP

# po limitach dopiero akceptacja SSH z dozwolonych źródeł
iptables -A INPUT -p tcp --dport 22 ! -s 192.168.1.55 -j ACCEPT
```

Uwaga: Limit musi być przed ogólnym ACCEPT tcp dpt:22, inaczej nigdy nie zadziała.

iptables: gotowe filtry do zadań

Blokuj wszystko wejściowe, ale zostaw SSH/ping/odpowiedzi

```
iptables -F
iptables -P INPUT DROP
iptables -P FORWARD ACCEPT
iptables -P OUTPUT ACCEPT
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

Test: Z innego hosta: ping działa; ssh działa; inne porty nie działają. Liczniki: iptables -L -n -v -x --line-numbers.

Zablokuj SSH z jednego adresu bez kasowania reguły

Użyj -I, bo reguły są od góry, a wcześniejszy ACCEPT ssh byłby silniejszy.

```
iptables -I INPUT 1 -p tcp -s 192.168.1.55 --dport 22 -j REJECT
# albo DROP, jeśli chcesz timeout zamiast szybkiej odmowy
```

Blokuj wyjście do WWW / hosta

```
iptables -A OUTPUT -p tcp -d www.cs.put.poznan.pl -j DROP
# gdy DNS zagrożenie na IP albo chcesz ograniczyć DNS:
iptables -A OUTPUT -p tcp -d 150.254.30.30 -j DROP
```

Uwaga: Reguła z nazwą DNS jest rozwijana w momencie dodawania reguły, nie dynamicznie przy każdym pakiecie.

Ruch tylko do jednej sieci dla grupy

```
iptables -A OUTPUT -m owner --gid-owner 1006 ! -d 192.168.1.0/24 -j DROP
```

Blokuj nowe TCP SYN, nie ruszając istniejących

```
iptables -A INPUT -p tcp --syn -j DROP
# równoważnie:
iptables -A INPUT -p tcp --tcp-flags SYN,RST,ACK,FIN SYN -j DROP
```

NAT: zasady i niuanse

Co zmienia SNAT/MASQUERADE/DNAT

Mechanizm	Gdzie	Sens
SNAT	nat POSTROUTING	Zmienia adres/port źródłowy pakietu wychodzącego. Używany, gdy znasz adres zewnętrzny routera.
MASQUERADE	nat POSTROUTING	SNAT automatycznie na adres interfejsu -o. Dobre dla DHCP/zmiennego IP.
DNAT	nat PREROUTING	Zmienia adres/port docelowy przed routingiem. Używany do przekierowania portu do hosta za NATem.

Najważniejsze niuanse NAT

- NAT nie zastępuje routingu.** Router musi mieć trasy, hosty muszą mieć bramę zwrotną, forwarding musi być włączony.
- SNAT i DNAT są niezależne.** SNAT: zmiana źródła w żądaniu, automatyczna zmiana celu w odpowiedzi. DNAT: zmiana celu w żądaniu, automatyczna zmiana źródła w odpowiedzi.
- NAT działa stanowo.** Conntrack pamięta mapowania, także translację portów, gdy dwa hosty prywatne użyją tego samego portu źródłowego.
- Zawężaj reguły.** Prawie zawsze dodaj -s, -d, -i, -o, -p, --dport. Zbyt ogólny SNAT na wszystkich interfejsach utrudnia debugowanie i może psuć komunikację lokalną.
- DNAT wymaga drogi powrotnej.** Host docelowy za NATem powinien mieć default przez router NAT albo trzeba dodać SNAT, żeby odpowiedź wróciła przez NAT.
- FORWARD może blokować DNAT.** Po DNAT pakiet jest routowany do hosta prywatnego, więc przechodzi przez FORWARD, nie INPUT.
- Lokalny test z routera używa OUTPUT, nie PREROUTING.** Typowe za-

danie testuje z innego PC, więc PREROUTING wystarczy.

Włączenie routera NAT

```
sysctl -w net.ipv4.ip_forward=1
# sprawdź trasy:
ip route
# sprawdź conntrack/reguły:
iptables -t nat -L -n -v -x --line-numbers
iptables -L FORWARD -n -v -x --line-numbers
```

SNAT i MASQUERADE - gotowce

Internet dla sieci prywatnej przez em1

```
# PC-router: p4p1 = LAN 192.168.1.1/24, em1 = internet/DHCP
sysctl -w net.ipv4.ip_forward=1
iptables -t nat -F
iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o em1 -j MASQUERADE
```

Na hostach LAN: `ip route add default via 192.168.1.1`. DNS: skopiuj/ustaw `/etc/resolv.conf`, jeśli ping po IP działa, a po nazwie nie.

Klasyczny SNAT na stały adres routera

```
iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o em1 -j SNAT --to-source 150.254.32.66
```

Użyj SNAT, gdy adres zewnętrzny jest znany i stały. Użyj MASQUERADE, gdy interfejs ma DHCP albo może zmienić adres.

SNAT między dwiema prywatnymi sieciami

Przykład: PC2 łączy PC1 192.168.1.0/24 i PC3 192.168.111.0/24. PC3 ma default do PC2, PC1 nie ma trasy do 192.168.111.0/24. Żeby PC3 mógł łączyć się z PC1 bez dodawania trasy na PC1:

```
# na PC2, interfejs do PC1 ma adres 192.168.1.2
iptables -t nat -A POSTROUTING \
-s 192.168.111.0/24 -d 192.168.1.0/24 \
-j SNAT --to-source 192.168.1.2
```

PC1 widzi ruch tak, jakby pochodził z PC2 192.168.1.2, więc odpowiada do PC2, a conntrack odtłumacza odpowiedź do PC3.

DNAT - przekierowanie portów

Prosty port forward do hosta za NATem

Założenie: PC1 łączy się z adresem PC2 od strony PC1, a usługa działa na PC3 192.168.111.1:74. PC2 ma forwarding.

```
# na PC2
iptables -t nat -A PREROUTING -i p4p1 -p tcp --dport 74 \
-j DNAT --to-destination 192.168.111.1:74
```

Test:

```
# PC3
ncat -l 192.168.111.1 74
# PC1 łączy się z adresem PC2 w sieci PC1-PC2
ncat 192.168.1.2 74
```

PC1 nie musi znać PC3. PC3 musi umieć odpowiedzieć przez PC2, np. default via 192.168.111.2.

Przekierowanie SSH na inny port

```
# PC2: port 1234 na PC2 -> SSH PC3:22
iptables -t nat -A PREROUTING -i p4p1 -p tcp --dport 1234 \
-j DNAT --to-destination 192.168.111.1:22
# PC1:
ssh -p 1234 user@192.168.1.2
```

Jeżeli FORWARD DROP, dodaj:

```
iptables -A FORWARD -i p4p1 -o p4p2 -p tcp -d 192.168.111.1 --dport 22 \
-m conntrack --ctstate NEW,ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -i p4p2 -o p4p1 -p tcp -s 192.168.111.1 --sport 22 \
-m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

DNAT bez trasy zwrotnej? Dodaj SNAT

Jeśli host docelowy nie ma defaultu przez router NAT, odpowiedź pójdzie inną drogą albo zginie. Wtedy możesz wymusić, żeby dla DNAT-owanych połączeń źródłem był adres routera od strony hosta prywatnego:

```
# DNAT z PC1 do PC3
iptables -t nat -A PREROUTING -i p4p1 -p tcp --dport 74 \
-j DNAT --to-destination 192.168.111.1:74
# SNAT odpowiedniego ruchu, żeby PC3 odpowiadał do PC2
iptables -t nat -A POSTROUTING -o p4p2 -p tcp -d 192.168.111.1 --dport 74 \
-j SNAT --to-source 192.168.111.2
```

Mangle i TTL

Ustawienie TTL dla ruchu przechodzącego przez router

Przykład z zadania NAT: pakiety z PC3 192.168.111.0/24 do PC1 192.168.1.0/24 mają wychodzić z PC2 z TTL=1.

```
iptables -t mangle -A POSTROUTING \
-s 192.168.111.0/24 -d 192.168.1.0/24 \
-j TTL --ttl-set 1
iptables -t mangle -L -n -v -x --line-numbers
```

Uwaga: W POSTROUTING ustawiasz TTL tuż przed wyjściem. Host docelowy może zobaczyć TTL=1; gdyby po drodze był kolejny router, pakiet by wygasł. Do obserwacji użyj Wiresharka.

Kompletny przykład NAT z trzema PC

Topologia

```
PC1 p4p1 192.168.1.1/24 --- p4p1 192.168.1.2/24 PC2
PC2 p4p2 192.168.111.2/24 --- p4p2 192.168.111.1/24 PC3
PC2 em1 Internet/DHCP
PC3 default via 192.168.111.2
PC2 forwarding = 1
```

PC1 może nie mieć defaultu, jeśli nie testujesz Internetu z PC1. PC3 powinien mieć default przez PC2.

Skrypt na PC2

```
#!/bin/bash
set -e
sysctl -w net.ipv4.ip_forward=1

iptables -P INPUT ACCEPT
iptables -P FORWARD ACCEPT
iptables -P OUTPUT ACCEPT
iptables -F
iptables -t nat -F
iptables -t mangle -F
iptables -X
iptables -Z

# Internet dla sieci za PC2 przez em1
iptables -t nat -A POSTROUTING -o em1 -j MASQUERADE

# PC3 -> PC1 bez trasy zwrotnej na PC1: PC1 widzi źródło 192.168.1.2
iptables -t nat -A POSTROUTING \
-s 192.168.111.0/24 -d 192.168.1.0/24 \
-j SNAT --to-source 192.168.1.2

# obserwacja TTL dla ruchu PC3 -> PC1
iptables -t mangle -A POSTROUTING \
-s 192.168.111.0/24 -d 192.168.1.0/24 \
-j TTL --ttl-set 1

# PC1 -> PC2:74 zostaje przekierowane na PC3:74
iptables -t nat -A PREROUTING -i p4p1 -p tcp --dport 74 \
-j DNAT --to-destination 192.168.111.1:74

# PC1 -> PC2:1234 zostaje przekierowane na PC3:ssh
iptables -t nat -A PREROUTING -i p4p1 -p tcp --dport 1234 \
-j DNAT --to-destination 192.168.111.1:22
```

Testy do tego przykładu

```
# PC2
iptables -t nat -L -n -v -x --line-numbers
iptables -t mangle -L -n -v -x --line-numbers

# PC3
ip route replace default via 192.168.111.2
ping 192.168.1.1 # PC1 zobaczy źródło 192.168.1.2 po SNAT
ncat -l 192.168.111.1 74
systemctl start sshd

# PC1
ncat 192.168.1.2 74
ssh -p 1234 user@192.168.1.2
```

W Wiresharku porównaj adresy na PC1, PC2-p4p1, PC2-p4p2, PC3. Przy SNAT zmienia się źródło; przy DNAT cel; odpowiedzi są odtłumaczane odwrotnie.

Typowe układy routingu na kolokwium

Dwa LAN-y przez jeden router Linux

```
PC1 192.168.10.2/24 -- p4p1 R 192.168.10.1/24
PC2 192.168.20.2/24 -- p4p2 R 192.168.20.1/24
```

```
# R
sysctl -w net.ipv4.ip_forward=1
# PC1
ip route add default via 192.168.10.1
# PC2
ip route add default via 192.168.20.1
```

Test: PC1 ping 192.168.20.2; traceroute pokaże router.

Łańcuch PC1-PC2-PC3

```
PC1 p4p1 10.0.12.1/24 -- 10.0.12.2/24 p4p1 PC2
PC2 p4p2 10.0.23.2/24 -- 10.0.23.3/24 p4p1 PC3
```

```
# PC2
sysctl -w net.ipv4.ip_forward=1
# PC1
ip route add 10.0.23.0/24 via 10.0.12.2
# PC3
ip route add 10.0.12.0/24 via 10.0.23.2
```

Alternatywnie defaulty na PC1 i PC3 do PC2.

Kilka sieci za jedną bramą

Jeśli za routerem R2 są 10.20.1.0/24, 10.20.2.0/24, ... i wszystko idzie przez ten sam next-hop, możesz dodać nadsieć, np. 10.20.0.0/16 via <R2>, o ile nie obejmuje to błędnych sieci w zadaniu. Default route jest jeszcze prostszy, jeśli cały nieznany ruch ma iść w tamtą stronę.

Wireshark, ping, traceroute - demonstracja

Co pokazać prowadzącemu

- **Routing:** ping między końcowymi hostami, traceroute -I, mtr, tablice ip route/show ip route.
- **Filtry:** ping działa, SSH nie działa albo odwrotnie; pokaż iptables -L -n -v -x --line-numbers; porównaj DROP vs REJECT w Wiresharku.
- **SNAT/MASQUERADE:** host prywatny wychodzi do innej sieci, ale z zewnątrz nie da się go bezpośrednio osiągnąć; w Wiresharku źródło po wyjściu z routera = adres routera.
- **DNAT:** nie łączysz się bezpośrednio z PC3, tylko z adresem/portem PC2, a trafiasz na usługę PC3.
- **VLAN:** hosty w różnych VLANach nie widzą broadcastów; po routerze między VLANami ping działa.

Traceroute i TTL

```
ping -c 3 192.168.20.2
ping -s 4000 192.168.20.2 # fragmentacja, do obserwacji
traceroute -I 192.168.20.2 # ICMP
traceroute -T 192.168.20.2 # TCP
tracert 192.168.20.2
mtr 192.168.20.2
```

Traceroute działa przez zwiększanie TTL i odpowiedzi ICMP Time Exceeded od kolejnych routerów. Brak odpowiedzi z hopa nie zawsze znaczy brak routingu - firewall może blokować ICMP.

Checklista „nie działa”

1. **Adresacja:** czy każda domena L2 ma inną sieć? Czy maski są takie same po obu stronach? Czy host nie ma starego adresu z poprzedniego zadania?
2. **Warstwa fizyczna:** czy port świeci? ip link/ethtool; Cisco show ip interface brief; właściwy kabel i patch panel?
3. **Lokalny ping:** najpierw ping tylko do bezpośredniego sąsiada. Jeśli nie działa, routing nie ma znaczenia - problem jest L1/L2/IP/maska/ARP.
4. **ARP:** ip neigh; czy ARP jest do sąsiada/bramy, a nie do zdalnego hosta?
5. **Forwarding:** na routerach Linux sysctl net.ipv4.ip_forward; na Cisco routing działa domyślnie między interfejsami L3.
6. **Trasy w obie strony:** źródło wie, gdzie wysłać? Każdy router wie, gdzie dalej? Cel wie, gdzie odesłać odpowiedź?
7. **iptables/NAT:** iptables -L -n -v -x, iptables -t nat -L -n -v -x; czy polityki nie blokują? Czy reguły są zbyt ogólne? Czy SNAT nie działa na złym interfejsie?
8. **Usługa:** netstat -tunap | grep PORT; czy słucha na właściwym IP? Czy port nie jest zajęty przez inny proces? Zabij proces lub zmień port.
9. **SSH:** systemctl status sshd; systemctl start sshd; ssh localhost; ssh -vv user@host.
10. **Wireshark:** sprawdź na każdym interfejsie, czy pakiet idzie w obie strony. Jedna strona bez odpowiedzi = routing powrotny, filtr albo NAT.

Mini-receptury końcowe

Komputer z Internetem dla dwóch hostów przez switch

Jeśli dwa hosty mają być w tej samej sieci prywatnej i wychodzić przez PC-router: hosty do switcha, router LAN do switcha, router WAN przez em1. Hosty default do adresu LAN routera; router ip_forward=1 i MASQUERADE -o em1. Jeśli hosty są w **różnych** sieciach, potrzebujesz routingu między tymi sieciami: albo dwa interfejsy routera, albo VLANy + router-on-a-stick, albo osobne routery.

Default route - kiedy używać

Użyj defaultu na hostach końcowych prawie zawsze: host zwykle zna tylko swoją sieć lokalną i bramę. Na routerach używaj defaultu, jeśli cały nieznany

ruch ma iść w jedną stronę. Nie dodawaj dwóch defaultów bez potrzeby, bo wybór zależy od metric/kolejności i utrudnia debugowanie.

Minimalny zapis konfiguracji na kartce

Dla każdej sieci zapisz: nazwa sieci / prefix, interfejsy, adresy, bramy. Dla każdego routera zapisz: trasy do sieci niebezpośrednich. Dla NAT/firewall zapisz: wejście/wyjście, źródło, cel, port, tabela/łańcuch.